SOP: Azure VM Image Capture with Compute Gallery (Nginx)

Oct 30, 2026 · @ashutosh

Overview

This SOP captures a running Nginx VM into an Azure Compute Gallery, rolls out three image versions (1.0.0 → 2.0.0 → 3.0.0), keeps only the newest marked as **latest**, and ends with a CLI command that always builds a VM from that latest version.

Scope: Linux VM (Ubuntu) serving `/var/www/html/index.html` through Nginx, Azure CLI 2.60+ (Bash / Azure Cloud Shell).

How "latest" works in Azure: when a VM is created from an image definition without a version (or with `/versions/latest`), Azure picks the **highest version number** whose `excludeFromLatest` flag is `false`. This SOP sets that flag explicitly on every version so the newest is always the one picked and older ones are never picked by accident.

Why Specialized images: a Specialized image is captured from a running VM without deprovisioning it. The source VM keeps serving traffic, and every VM built from the image boots with the same Nginx config and content. A Generalized image would require `waagent -deprovision` and `az vm generalize`, which permanently retires the source VM.

Version	Built from	index.html shows	excludeFromLatest at the end
1.0.0	Original VM `vm-nginx-src`	Version 1 page	true
2.0.0	`vm-web-v1` (edited)	Version 2 page	true
3.0.0	`vm-web-v2` (edited)	Version 3 page	**false (LATEST)**

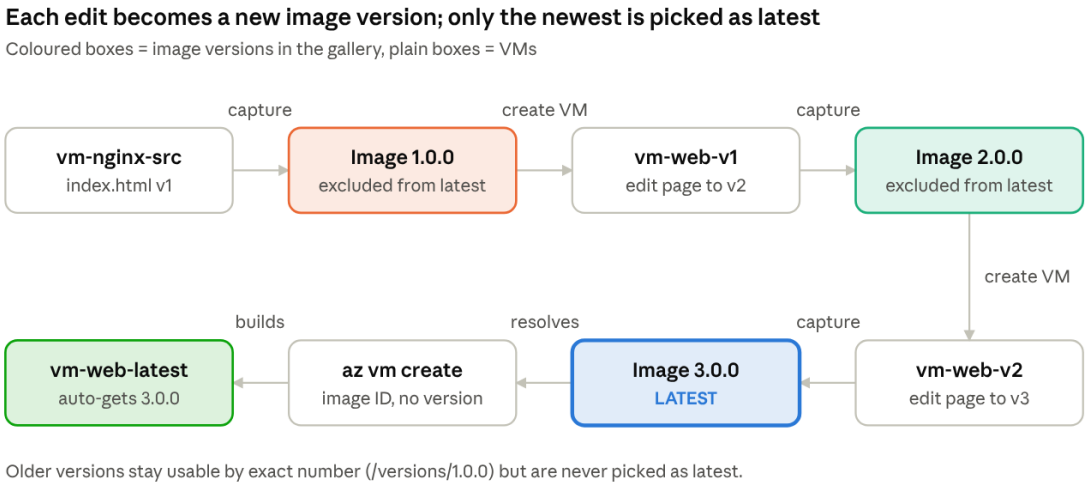


image lifecycle · 3 versions, 1 latest

Each round builds a VM from the previous version, edits the page, and captures the next version; the final `az vm create` always lands on the newest.

Step 0 · Variables and prerequisites

Set these once per shell session; every later command reuses them. Change the values to match your environment.

```
# ---- Login & subscription ----
az login
az account set --subscription "<SUBSCRIPTION_NAME_OR_ID>"

# ---- Common variables ----
RG="rg-nginx-images"            # resource group of the source VM
LOCATION="centralindia"         # same region as the source VM
SRC_VM="vm-nginx-src"           # VM already running Nginx + index.html
GALLERY="galNginxImages"        # gallery name: letters, numbers, dots, underscores
only
IMG_DEF="nginx-web"             # image definition (the image 'family')
VM_SIZE="Standard_B1s"
```

Prerequisites checklist:

- ☐ Azure CLI 2.60 or newer (`az version`)
- ☐ Contributor rights on the resource group
- ☐ Source VM is running and Nginx serves `index.html` on port 80
- ☐ Port 22 (SSH) or Run Command access to the VMs
- ☐ Know the source VM's **Hyper-V generation** and **security type** (checked in Step 1)

Step 1 · Verify the source VM

Confirm the page is live and read the two properties the image definition must match.

1. Confirm Nginx serves the page:

```
SRC_IP=$(az vm show -d -g $RG -n $SRC_VM --query publicIps -o tsv)
curl -s http://$SRC_IP | head -n 5
```

2. Read the Hyper-V generation (V1 or V2):

```
az vm get-instance-view -g $RG -n $SRC_VM \
  --query "instanceView.hyperVGeneration" -o tsv
```

3. Read the security type (empty = Standard, otherwise TrustedLaunch):

```
az vm show -g $RG -n $SRC_VM \
  --query "securityProfile.securityType" -o tsv
```

4. Store the results for the next steps:

```
HYPERV_GEN="V2"                 # value from command 2
SECURITY_TYPE="TrustedLaunch"   # value from command 3 (use Standard if empty)
```

> ***Important:*** *a mismatch in generation or security type makes the image version creation fail. Always copy the real values.*

Step 2 · Create the Compute Gallery and image definition

The gallery is the container; the image definition is the family that every version (1.0.0, 2.0.0, 3.0.0) belongs to.

1. Create the Azure Compute Gallery:

```
az sig create \
  --resource-group $RG \
  --gallery-name $GALLERY \
  --location $LOCATION \
  --description "Nginx web server golden images"
```

2. Create the image definition (Specialized, matching the source VM):

```
# Only add the TrustedLaunch feature when the source VM uses it
FEATURES=""
[ "$SECURITY_TYPE" = "TrustedLaunch" ] && FEATURES="--features
SecurityType=TrustedLaunch"

az sig image-definition create \
  --resource-group $RG \
  --gallery-name $GALLERY \
  --gallery-image-definition $IMG_DEF \
  --publisher MyOrg \
  --offer NginxWeb \
  --sku Ubuntu-Nginx \
  --os-type Linux \
  --os-state Specialized \
  --hyper-v-generation $HYPERV_GEN \
  $FEATURES
```

3. Save the definition ID; every later command uses it:

```
IMG_DEF_ID=$(az sig image-definition show \
  -g $RG -r $GALLERY -i $IMG_DEF --query id -o tsv)
echo $IMG_DEF_ID
```

Expected result: `provisioningState` is `Succeeded` for both the gallery and the definition.

Step 3 · Capture image version 1.0.0 from the source VM

Version 1.0.0 is captured straight from the running VM and marked as latest (it is the only one so far).

```
SRC_VM_ID=$(az vm show -g $RG -n $SRC_VM --query id -o tsv)

az sig image-version create \
  --resource-group $RG \
  --gallery-name $GALLERY \
  --gallery-image-definition $IMG_DEF \
  --gallery-image-version 1.0.0 \
  --virtual-machine $SRC_VM_ID \
  --target-regions $LOCATION \
  --replica-count 1 \
  --exclude-from-latest false
```

> ***Tip:*** *capture takes 10 to 30 minutes. Stop writes on the VM (or briefly stop Nginx) for a clean, consistent snapshot.*

Check it finished:

```
az sig image-version show -g $RG -r $GALLERY -i $IMG_DEF -e 1.0.0 \
  --query "{version:name, state:provisioningState,
excluded:publishingProfile.excludeFromLatest}" -o table
```

Step 4 · Round 1: VM from 1.0.0 → edit index.html → version 2.0.0 (latest)

Build a VM from 1.0.0, change the page, capture it as 2.0.0, then hide 1.0.0 from "latest".

4.1 Create a VM from version 1.0.0

```
SEC_ARGS=""
[ "$SECURITY_TYPE" = "TrustedLaunch" ] && SEC_ARGS="--security-type TrustedLaunch"

az vm create \
  --resource-group $RG \
  --name vm-web-v1 \
  --image "$IMG_DEF_ID/versions/1.0.0" \
  --specialized \
  --size $VM_SIZE \
  --public-ip-sku Standard \
  $SEC_ARGS

az vm open-port -g $RG -n vm-web-v1 --port 80 --priority 1010
```

> *Specialized images keep the original OS user and SSH keys, so no `--admin-username` or `--ssh-key-values` is passed.*

4.2 Update index.html to Version 2

Run Command edits the file without SSH; you can also SSH in and use `sudo nano /var/www/html/index.html`.

```
az vm run-command invoke -g $RG -n vm-web-v1 \
  --command-id RunShellScript \
  --scripts 'cat <<EOF | sudo tee /var/www/html/index.html
<!DOCTYPE html>
<html><head><title>Nginx Image v2</title></head>
<body style="font-family:Arial;background:#0f766e;color:#fff;text-
align:center;padding-top:80px">
  <h1>Version 2.0.0</h1>
  <p>Served from image nginx-web 2.0.0</p>
</body></html>
EOF
sudo nginx -t && sudo systemctl reload nginx'

V1_IP=$(az vm show -d -g $RG -n vm-web-v1 --query publicIps -o tsv)
curl -s http://$V1_IP | grep "<h1>"     # expect: Version 2.0.0
```

4.3 Capture version 2.0.0 and mark it latest

```
V1_ID=$(az vm show -g $RG -n vm-web-v1 --query id -o tsv)

az sig image-version create \
  --resource-group $RG \
```

```
  --gallery-name $GALLERY \
  --gallery-image-definition $IMG_DEF \
  --gallery-image-version 2.0.0 \
  --virtual-machine $V1_ID \
  --target-regions $LOCATION \
  --replica-count 1 \
  --exclude-from-latest false
```

4.4 Remove "latest" from version 1.0.0

```
az sig image-version update \
  -g $RG -r $GALLERY -i $IMG_DEF -e 1.0.0 \
  --set publishingProfile.excludeFromLatest=true
```

Result: 2.0.0 = latest, 1.0.0 = still usable by exact version, never picked as latest.

Step 5 · Round 2: VM from 2.0.0 → edit index.html → version 3.0.0 (latest)

Same cycle once more; 3.0.0 becomes the latest and 2.0.0 is hidden from "latest".

5.1 Create a VM from version 2.0.0

```
az vm create \
  --resource-group $RG \
  --name vm-web-v2 \
  --image "$IMG_DEF_ID/versions/2.0.0" \
  --specialized \
  --size $VM_SIZE \
  --public-ip-sku Standard \
  $SEC_ARGS

az vm open-port -g $RG -n vm-web-v2 --port 80 --priority 1010
```

5.2 Update index.html to Version 3

```
az vm run-command invoke -g $RG -n vm-web-v2 \
  --command-id RunShellScript \
  --scripts 'cat <<EOF | sudo tee /var/www/html/index.html
<!DOCTYPE html>
<html><head><title>Nginx Image v3</title></head>
<body style="font-family:Arial;background:#7c3aed;color:#fff;text-
align:center;padding-top:80px">
  <h1>Version 3.0.0</h1>
  <p>Served from image nginx-web 3.0.0 (LATEST)</p>
</body></html>
EOF
sudo nginx -t && sudo systemctl reload nginx'

V2_IP=$(az vm show -d -g $RG -n vm-web-v2 --query publicIps -o tsv)
curl -s http://$V2_IP | grep "<h1>"     # expect: Version 3.0.0
```

5.3 Capture version 3.0.0 and mark it latest

```
V2_ID=$(az vm show -g $RG -n vm-web-v2 --query id -o tsv)

az sig image-version create \
```

```
  --resource-group $RG \
  --gallery-name $GALLERY \
  --gallery-image-definition $IMG_DEF \
  --gallery-image-version 3.0.0 \
  --virtual-machine $V2_ID \
  --target-regions $LOCATION \
  --replica-count 1 \
  --exclude-from-latest false
```

5.4 Remove "latest" from version 2.0.0

```
az sig image-version update \
  -g $RG -r $GALLERY -i $IMG_DEF -e 2.0.0 \
  --set publishingProfile.excludeFromLatest=true
```

Result: 3.0.0 = latest; 1.0.0 and 2.0.0 are both excluded.

Step 6 · Verify which version is latest

List every version with its latest flag; only 3.0.0 should show `False`.

```
az sig image-version list \
  -g $RG -r $GALLERY -i $IMG_DEF \
  --query "[].{Version:name, ExcludedFromLatest:publishingProfile.excludeFromLatest,
State:provisioningState, Created:publishingProfile.publishedDate}" \
  -o table
```

Expected output:

```
Version    ExcludedFromLatest    State      Created
---------  --------------------  ---------  -------------------------
1.0.0      True                  Succeeded  2026-10-30T07:10:11+00:00
2.0.0      True                  Succeeded  2026-10-30T08:02:45+00:00
3.0.0      False                 Succeeded  2026-10-30T08:55:19+00:00
```

Print only the version Azure will treat as latest:

```
az sig image-version list -g $RG -r $GALLERY -i $IMG_DEF \
  --query "[?publishingProfile.excludeFromLatest==\`false\`].name | sort(@) | [-1]" -
o tsv
```

> *`sort()` here is alphabetical, which is correct while versions keep the same digit pattern (1.0.0 … 9.0.0). Azure itself always compares versions numerically.*

Step 7 · Create a VM that always uses the LATEST image

Point `--image` at the image definition ID with **no version number**; Azure resolves it to the current latest (3.0.0 today, 4.0.0 tomorrow) with no script change.

Option A · One command

```
az vm create \
  --resource-group $RG \
  --name vm-web-latest \
  --image "$IMG_DEF_ID" \
  --specialized \
  --size $VM_SIZE \
```

```
  --public-ip-sku Standard \
  $SEC_ARGS
```

`--image "$IMG_DEF_ID/versions/latest"` gives the same result if you prefer the version to be visible.

Option B · Reusable script (create-latest-vm.sh)

```
#!/usr/bin/env bash
set -euo pipefail

RG="rg-nginx-images"
GALLERY="galNginxImages"
IMG_DEF="nginx-web"
VM_NAME="${1:-vm-web-$(date +%Y%m%d%H%M)}"

IMG_DEF_ID=$(az sig image-definition show -g $RG -r $GALLERY -i $IMG_DEF --query id -
o tsv)
SEC=$(az sig image-definition show -g $RG -r $GALLERY -i $IMG_DEF \
  --query "features[?name=='SecurityType'].value | [0]" -o tsv)
SEC_ARGS=""; [ "$SEC" = "TrustedLaunch" ] && SEC_ARGS="--security-type TrustedLaunch"

echo "Creating $VM_NAME from the latest version of $IMG_DEF ..."
az vm create -g $RG -n "$VM_NAME" \
  --image "$IMG_DEF_ID" --specialized \
  --size Standard_B1s --public-ip-sku Standard $SEC_ARGS -o none

az vm open-port -g $RG -n "$VM_NAME" --port 80 --priority 1010 -o none

VER=$(az vm show -g $RG -n "$VM_NAME" --query
storageProfile.imageReference.exactVersion -o tsv)
IP=$(az vm show -d -g $RG -n "$VM_NAME" --query publicIps -o tsv)
echo "Done: $VM_NAME built from image version $VER -> http://$IP"
```

Run it:

```
chmod +x create-latest-vm.sh
./create-latest-vm.sh vm-web-prod01
```

Confirm the VM got the latest version

```
az vm show -g $RG -n vm-web-latest \
  --query storageProfile.imageReference.exactVersion -o tsv   # expect: 3.0.0
curl -s http://$(az vm show -d -g $RG -n vm-web-latest --query publicIps -o tsv) |
grep "<h1>"
```

Troubleshooting

Symptom	Likely cause	Fix
`The specified hyperVGeneration ... does not match`	Definition generation differs from the VM	Recreate the definition with the value from Step 1
`SecurityType ... TrustedLaunch` error on	Definition lacks the TrustedLaunch feature	Recreate the definition with `--features`

Symptom	Likely cause	Fix
version create		`SecurityType=TrustedLaunch`
VM from image asks for admin credentials	`--specialized` was omitted	Add `--specialized`; drop `--admin-username` / SSH key flags
New VM uses an old page	An older version still has `excludeFromLatest=false`	Re-run Step 6 and set the old version to `true`
Page not reachable on new VM	Port 80 closed in the NSG	`az vm open-port --port 80`
Version create stuck for long	Large OS disk or many replicas	Wait; check `provisioningState`, keep `--replica-count 1` for labs

Rollback and cleanup

Roll back "latest" to 2.0.0 without deleting anything:

```
az sig image-version update -g $RG -r $GALLERY -i $IMG_DEF -e 3.0.0 --set
publishingProfile.excludeFromLatest=true
az sig image-version update -g $RG -r $GALLERY -i $IMG_DEF -e 2.0.0 --set
publishingProfile.excludeFromLatest=false
```

Delete lab resources when finished (versions must go before the definition, the definition before the gallery):

```
az vm delete -g $RG -n vm-web-v1 --yes
az vm delete -g $RG -n vm-web-v2 --yes
az vm delete -g $RG -n vm-web-latest --yes
for v in 1.0.0 2.0.0 3.0.0; do az sig image-version delete -g $RG -r $GALLERY -i
$IMG_DEF -e $v; done
az sig image-definition delete -g $RG -r $GALLERY -i $IMG_DEF
az sig delete -g $RG -r $GALLERY
```

> *`az vm delete` leaves NICs, public IPs and disks behind; remove them too, or delete the whole resource group if it holds only lab resources.*